

EDUCATIONAL PDF NOTES

LeetCode Pattern Notes

25 problems organized by topic, with clean code, side-by-side syntax explanations, and beginner-friendly summaries for revision and sharing.

Praveen Mosesraj Solomon
Software Engineer

CREDIT AND PURPOSE

Learning notes for structured LeetCode revision

This PDF is prepared as a clean educational reference for the 25 LeetCode problems from Praveen's study files. It is designed for revision, pattern recognition, and beginner-friendly explanation.

Prepared for

Praveen Mosesraj Solomon

Professional title

Software Engineer

Scope

Arrays & Hashing, Two Pointers, Stack, Binary Search, Sliding Window

Format

Code + syntax meaning + plain English explanation

Credit note: Created and credited to Praveen Mosesraj Solomon for educational use, personal revision, and non-commercial sharing. This is not a strict copyright notice; it is a professional credit line for ownership of the learning compilation.

25 Problems by Topic and Pattern

Use this page as the quick roadmap before reading the detailed problem notes.

5

ARRAYS

5

TWO POINTERS

5

STACK

5

BINARY SEARCH

5

SLIDING WINDOW

No.	Topic	Problem	Pattern / Method
1	Arrays & Hashing	Two Sum	Brute Force / Pair Checking
2	Arrays & Hashing	Contains Duplicate	Hash Set
3	Arrays & Hashing	Valid Anagram	Frequency Map / Sorting
4	Arrays & Hashing	Group Anagrams	Hash Map with Sorted String Key
5	Arrays & Hashing	Top K Frequent Elements	Frequency Map + Sorting
6	Two Pointers	Valid Palindrome	String Cleaning / Two Pointer Idea
7	Two Pointers	3Sum	Brute Force and Sort + Two Pointers
8	Two Pointers	Container With Most Water	Two Pointers
9	Two Pointers	Trapping Rain Water	Two Pointers with Left Max and Right Max
10	Two Pointers	Two Sum II - Input Array Is Sorted	Two Pointers on Sorted Array
11	Stack	Valid Parentheses	Stack Matching
12	Stack	Min Stack	Two Stacks
13	Stack	Daily Temperatures	Monotonic Stack
14	Stack	Generate Parentheses	Backtracking
15	Stack	Largest Rectangle in Histogram	Monotonic Increasing Stack
16	Binary Search	Binary Search	Classic Binary Search
17	Binary Search	Search in Rotated Sorted Array	Modified Binary Search
18	Binary Search	Find Minimum in Rotated Sorted Array	Modified Binary Search
19	Binary Search	Koko Eating Bananas	Binary Search on Answer

No.	Topic	Problem	Pattern / Method
20	Binary Search	Median of Two Sorted Arrays	Merge + Sort Simple Approach
21	Sliding Window	Best Time to Buy and Sell Stock	Sliding Window / Two Pointers
22	Sliding Window	Longest Substring Without Repeating Characters	Sliding Window + Hash Set
23	Sliding Window	Minimum Window Substring	Sliding Window + Counter
24	Sliding Window	Longest Repeating Character Replacement	Sliding Window + Frequency Map
25	Sliding Window	Sliding Window Maximum	Brute Force and Monotonic Deque

TOPIC

Arrays & Hashing

Problems 1 to 5

Two Sum

Purpose

Find two different indexes whose values add up to the target.

Complexity

Time: $O(n^2)$, Space: $O(1)$

Clean code

```
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        index1 = 0
        index2 = 0

        for i in range(len(nums)):
            numc1 = nums[index1]

            for eachnum in nums:
                if numc1 + eachnum == target and index1 != index2:
                    return [index1, index2]

            index2 += 1

        index1 += 1
        index2 = 0
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Typing import	from typing import List	Imports List so type hints like List[int] can be used.
Class wrapper	class Solution:	LeetCode expects the answer inside a Solution class.
Function definition	def twoSum(self, nums: List[int], target: int) -> List[int]:	Receives the number list and target, then returns two indexes.
First pointer/index	index1 = 0	Starts the first number at index 0.
Second pointer/index	index2 = 0	Starts the second number at index 0 for checking pairs.
Outer loop	for i in range(len(nums)):	Runs once for each possible first number.
Current first value	numc1 = nums[index1]	Gets the value at the first index.
Inner loop	for eachnum in nums:	Checks the first value against every number in the array.
Target check	if numc1 + eachnum == target and index1 != index2:	If both values add to target and are not the same index, the pair is valid.
Return answer	return [index1, index2]	Returns the two indexes immediately.
Move second index	index2 += 1	Moves to the next possible second number.
Move first index	index1 += 1	Moves to the next possible first number.

Syntax / concept	Actual code line	Plain English meaning
Reset second index	<code>index2 = 0</code>	Starts the second index again from the beginning for the next first number.

Final summary

This version tries every pair of numbers. It works because every possible pair is tested, but it is slower than the hash map approach because it checks many repeated combinations.

Contains Duplicate

Purpose

Check whether any value appears at least twice.

Complexity

Time: $O(n)$, Space: $O(n)$

Clean code

```
from typing import List

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        duplicates = len(set(nums)) != len(nums)
        return duplicates
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Function definition	def containsDuplicate(self, nums: List[int]) -> bool:	Takes a list of numbers and returns True or False.
Convert to set	set(nums)	A set removes duplicate values automatically.
Compare lengths	len(set(nums)) != len(nums)	If the set length is smaller than the original list length, duplicates existed.
Store boolean	duplicates = ...	Saves the True/False result in a variable.
Return result	return duplicates	Sends the final answer back.

Final summary

The set keeps only unique values. If the unique count is different from the original count, at least one duplicate was present.

Valid Anagram

Purpose

Check whether two strings contain the same characters with the same counts.

Complexity

Map: $O(n)$, Sorting: $O(n \log n)$

Clean code

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        freq = {}

        for char in s:
            if char in freq:
                freq[char] += 1
            else:
                freq[char] = 1

        for char in t:
            if char in freq:
                freq[char] -= 1
            else:
                freq[char] = -1

        for value in freq.values():
            if value != 0:
                return False

        return True

    def isAnagram2(self, s: str, t: str) -> bool:
        return sorted(s) == sorted(t)
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Dictionary setup	<code>freq = {}</code>	Creates a dictionary to count character frequency.
Loop through s	<code>for char in s:</code>	Reads every character in the first string.
Existing key check	<code>if char in freq:</code>	Checks if this character is already counted.
Increase count	<code>freq[char] += 1</code>	Adds 1 for the character from s.
First time count	<code>freq[char] = 1</code>	Starts the count when the character appears for the first time.
Loop through t	<code>for char in t:</code>	Reads every character in the second string.
Decrease count	<code>freq[char] -= 1</code>	Subtracts 1 because t should cancel the count from s.
Missing character	<code>freq[char] = -1</code>	If t has a new character, the balance becomes negative.
Check all balances	<code>for value in freq.values():</code>	Looks at every final character count.
Mismatch check	<code>if value != 0:</code>	Any non-zero count means the strings differ.

Syntax / concept	Actual code line	Plain English meaning
Fail result	<code>return False</code>	Returns False when counts are not balanced.
Success result	<code>return True</code>	Returns True when every count is zero.
Sorting shortcut	<code>return sorted(s) == sorted(t)</code>	If sorted characters match, both strings are anagrams.

Final summary

The frequency method adds counts from the first string and subtracts counts from the second. If all counts become zero, both strings contain exactly the same letters.

Group Anagrams

Purpose

Group words that become the same when their letters are sorted.

Complexity

Time: $O(n * k \log k)$, Space: $O(n * k)$

Clean code

```
from typing import List

class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        anagram_groups = {}

        for word in strs:
            sorted_word = ''.join(sorted(word))

            if sorted_word in anagram_groups:
                anagram_groups[sorted_word].append(word)
            else:
                anagram_groups[sorted_word] = [word]

        return list(anagram_groups.values())
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Dictionary setup	<code>anagram_groups = {}</code>	Stores each sorted pattern and its matching words.
Loop words	<code>for word in strs:</code>	Processes one word at a time.
Sort letters	<code>sorted(word)</code>	Sorts the letters so anagrams get the same order.
Join letters	<code>''.join(sorted(word))</code>	Turns the sorted character list back into a string key.
Key exists check	<code>if sorted_word in anagram_groups:</code>	Checks if this anagram group already exists.
Append to group	<code>anagram_groups[sorted_word].append(word)</code>	Adds the current word to the existing group.
Create new group	<code>anagram_groups[sorted_word] = [word]</code>	Starts a new list for this sorted key.
Return groups	<code>return list(anagram_groups.values())</code>	Returns only the grouped word lists, not the keys.

Final summary

Every anagram has the same sorted form. The sorted form becomes the dictionary key, and all matching words are stored under that key.

Top K Frequent Elements

Purpose

Return the k numbers that appear most frequently.

Complexity

Time: $O(n \log n)$, Space: $O(n)$

Clean code

```
from typing import List

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        element_freq = {}

        for num in nums:
            if num in element_freq:
                element_freq[num] += 1
            else:
                element_freq[num] = 1

        sorted_freq = sorted(
            element_freq.items(),
            key=lambda x: x[1],
            reverse=True
        )

        top_k_elements = [item[0] for item in sorted_freq[:k]]
        return top_k_elements
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Frequency dictionary	<code>element_freq = {}</code>	Stores each number and how many times it appears.
Loop numbers	<code>for num in nums:</code>	Reads every number.
Existing number	<code>if num in element_freq:</code>	Checks whether this number already has a count.
Increase count	<code>element_freq[num] += 1</code>	Adds 1 to the existing frequency.
Start count	<code>element_freq[num] = 1</code>	Sets count to 1 for a new number.
Dictionary items	<code>element_freq.items()</code>	Creates pairs like (number, frequency).
Sort by frequency	<code>key=lambda x: x[1]</code>	Sorts using the second item in each pair, which is the frequency.
Descending order	<code>reverse=True</code>	Places highest frequency first.
Take first k	<code>sorted_freq[:k]</code>	Keeps only the top k frequency pairs.
Extract numbers	<code>[item[0] for item in sorted_freq[:k]]</code>	Gets only the number from each pair, not the frequency.
Return answer	<code>return top_k_elements</code>	Returns the k most frequent numbers.

Final summary

First count how often every number appears. Then sort the number-frequency pairs by frequency and return the first k numbers.

TOPIC

Two Pointers

Problems 6 to 10

Valid Palindrome

Purpose

Check if a cleaned string reads the same forward and backward.

Complexity

Time: $O(n)$, Space: $O(n)$

Clean code

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        cleaned = ''.join(filter(str.isalnum, s)).lower()
        return cleaned == cleaned[::-1]
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Function definition	def isPalindrome(self, s: str) -> bool:	Takes a string and returns whether it is a palindrome.
Filter characters	filter(str.isalnum, s)	Keeps only letters and numbers.
Join result	''.join(...)	Combines the filtered characters into one string.
Lowercase	.lower()	Makes comparison case-insensitive.
Reverse slicing	cleaned[::-1]	Creates the reversed version of the cleaned string.
Compare strings	cleaned == cleaned[::-1]	True if forward and backward strings are the same.

Final summary

The code removes punctuation/spaces, lowercases the text, and compares it with its reverse. If both match, the string is a valid palindrome.

3Sum

Brute Force and Sort + Two Pointers

Purpose
Find unique triplets that add up to zero.

Complexity
Brute: $O(n^3)$. Optimized: $O(n^2)$

Clean code

```
class Solution:
    def threeSumBrute(self, nums: list[int]) -> list[list[int]]:
        no_duplicates = set()

        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                for k in range(j + 1, len(nums)):
                    if nums[i] + nums[j] + nums[k] == 0:
                        triplet = sorted([nums[i], nums[j], nums[k]])
                        no_duplicates.add(tuple(triplet))

        return [list(triplet) for triplet in no_duplicates]

    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        result = []

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            left = i + 1
            right = len(nums) - 1

            while left < right:
                total = nums[i] + nums[left] + nums[right]

                if total < 0:
                    left += 1
                elif total > 0:
                    right -= 1
                else:
                    result.append([nums[i], nums[left], nums[right]])
                    left += 1
                    right -= 1

                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1

            return result
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Duplicate protection	no_duplicates = set()	Stores triplets as tuples so repeated triplets are removed.
Three loops	for i / for j / for k	Brute force checks every possible group of three indexes.
Zero check	if nums[i] + nums[j] + nums[k] == 0:	Accepts only triplets whose sum is zero.

Syntax / concept	Actual code line	Plain English meaning
Sort triplet	<code>sorted([nums[i], nums[j], nums[k]])</code>	Makes the same triplet order consistent before storing.
Tuple in set	<code>no_duplicates.add(tuple(triplet))</code>	Tuples can be stored in a set; lists cannot.
Sort array	<code>nums.sort()</code>	Sorting allows the two pointer strategy.
Main index loop	<code>for i in range(len(nums) - 2):</code>	Fixes one number and leaves two positions for left and right.
Skip duplicate i	<code>if i > 0 and nums[i] == nums[i - 1]: continue</code>	Avoids creating the same triplet again.
Left pointer	<code>left = i + 1</code>	Starts just after the fixed number.
Right pointer	<code>right = len(nums) - 1</code>	Starts at the last index.
Two pointer loop	<code>while left < right:</code>	Keeps checking until both pointers meet.
Sum calculation	<code>total = nums[i] + nums[left] + nums[right]</code>	Calculates the current triplet sum.
Need bigger sum	<code>if total < 0: left += 1</code>	Move left rightward to increase the sum.
Need smaller sum	<code>elif total > 0: right -= 1</code>	Move right leftward to decrease the sum.
Found triplet	<code>result.append(...)</code>	Saves the valid triplet.
Move both pointers	<code>left += 1; right -= 1</code>	Moves away from the used pair.
Skip duplicates	<code>while ... nums[left] == nums[left - 1]</code>	Skips repeated left values after a valid triplet.

Final summary

The brute force method tries all triplets. The optimized method sorts the array, fixes one number, and uses left/right pointers to find the remaining two numbers efficiently.

Container With Most Water

Purpose

Find the maximum water area between two vertical lines.

Complexity

Time: O(n), Space: O(1)

Clean code

```
from typing import List

class Solution:
    def maxArea(self, height: List[int]) -> int:
        left_index = 0
        right_index = len(height) - 1
        max_area = 0

        while left_index < right_index:
            width = right_index - left_index
            current_height = min(height[left_index], height[right_index])
            current_area = current_height * width
            max_area = max(current_area, max_area)

            if height[left_index] < height[right_index]:
                left_index += 1
            else:
                right_index -= 1

        return max_area
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Left pointer	left_index = 0	Starts at the first line.
Right pointer	right_index = len(height) - 1	Starts at the last line.
Best answer	max_area = 0	Stores the biggest area found so far.
Loop until meet	while left_index < right_index:	Checks containers while there are two different lines.
Width	width = right_index - left_index	Distance between the two selected lines.
Container height	min(height[left_index], height[right_index])	Water height is limited by the shorter line.
Area	current_area = current_height * width	Area equals height times width.
Update best	max_area = max(current_area, max_area)	Keeps the larger area.
Move shorter side	if height[left_index] < height[right_index]: left_index += 1	Move the shorter side because it is limiting the water.
Move right side	else: right_index -= 1	When right is shorter or equal, move it inward.
Return result	return max_area	Returns the maximum container area.

Final summary

The area is limited by the shorter wall. Moving the taller wall cannot help if the shorter wall stays, so the algorithm always moves the shorter side inward.

Trapping Rain Water

Purpose

Calculate how much water can be trapped between bars.

Complexity

Time: O(n), Space: O(1)

Clean code

```
from typing import List

class Solution:
    def trap(self, height: List[int]) -> int:
        left_index = 0
        right_index = len(height) - 1
        left_max = 0
        right_max = 0
        total_trapped_water = 0

        while left_index < right_index:
            if height[left_index] < height[right_index]:
                if height[left_index] >= left_max:
                    left_max = height[left_index]
                else:
                    total_trapped_water += left_max - height[left_index]
                    left_index += 1
            else:
                if height[right_index] >= right_max:
                    right_max = height[right_index]
                else:
                    total_trapped_water += right_max - height[right_index]
                    right_index -= 1

        return total_trapped_water
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Two ends	left_index = 0; right_index = len(height) - 1	Start checking from both sides.
Max walls	left_max = 0; right_max = 0	Track the tallest wall seen from each side.
Water total	total_trapped_water = 0	Stores the total trapped water.
Loop	while left_index < right_index:	Continue until both pointers meet.
Choose smaller side	if height[left_index] < height[right_index]:	The smaller side decides the trapped water at that position.
Update left max	left_max = height[left_index]	If current left wall is taller, update left max.
Trap left water	total_trapped_water += left_max - height[left_index]	If current wall is lower, water is trapped above it.
Move left	left_index += 1	Move inward from the left side.
Update right max	right_max = height[right_index]	If current right wall is taller, update right max.

Syntax / concept	Actual code line	Plain English meaning
Trap right water	<code>total_trapped_water += right_max - height[right_index]</code>	If current right wall is lower, water is trapped above it.
Move right	<code>right_index -= 1</code>	Move inward from the right side.
Return total	<code>return total_trapped_water</code>	Returns total trapped units of water.

Final summary

At each step, the shorter side is safe to process because the opposite side already has a boundary tall enough. The difference between max wall and current height gives trapped water.

Two Sum II - Input Array Is Sorted

Purpose

Find two 1-based indexes in a sorted array whose sum equals target.

Complexity

Time: $O(n)$, Space: $O(1)$

Clean code

```
from typing import List

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        left = 0
        right = len(numbers) - 1

        while left < right:
            total = numbers[left] + numbers[right]

            if total > target:
                right -= 1
            elif total < target:
                left += 1
            else:
                return [left + 1, right + 1]
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Left pointer	left = 0	Starts at the smallest number.
Right pointer	right = len(numbers) - 1	Starts at the largest number.
Loop	while left < right:	Checks pairs until pointers meet.
Current sum	total = numbers[left] + numbers[right]	Adds the current two numbers.
Too large	if total > target: right -= 1	Move right leftward to make the sum smaller.
Too small	elif total < target: left += 1	Move left rightward to make the sum larger.
Found answer	return [left + 1, right + 1]	Returns 1-based positions as LeetCode asks.

Final summary

Because the array is sorted, moving left increases the sum and moving right decreases the sum. That lets us find the pair in one pass.

TOPIC

Stack

Problems 11 to 15

Valid Parentheses

Purpose
Check if every closing bracket matches the most recent opening bracket.

Complexity
Time: O(n), Space: O(n)

Clean code

```
class Solution:
    def isValid(self, s: str) -> bool:
        if len(s.strip()) % 2 != 0:
            return False

        pairs = {
            ')': '(',
            ']': '[',
            '}': '{'
        }
        stack = []

        for char in s:
            if char in ('(', '[', '{'):
                stack.append(char)
            else:
                if not stack:
                    return False
                if stack[-1] != pairs[char]:
                    return False
                stack.pop()

        return not stack
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Odd length check	if len(s.strip()) % 2 != 0:	A valid bracket string must have an even number of brackets.
Pair map	pairs = {')': '(', ']': '[', '}': '{'}	Maps every closing bracket to its matching opening bracket.
Stack setup	stack = []	Stores opening brackets that have not been closed yet.
Loop characters	for char in s:	Reads each bracket one by one.
Opening bracket	if char in ('(', '[', '{):	If it is an opening bracket, save it.
Push stack	stack.append(char)	Adds the opening bracket to the top of the stack.
Empty stack fail	if not stack: return False	A closing bracket cannot be valid if there is no opening bracket.
Top match check	stack[-1] != pairs[char]	Checks whether the latest opening bracket matches the current closing bracket.
Pop matched pair	stack.pop()	Removes the matched opening bracket.

Syntax / concept	Actual code line	Plain English meaning
Final check	return not stack	True only if no unmatched openings remain.

Final summary

Stacks follow last-in-first-out. The most recent opening bracket must be the first one closed, so `stack[-1]` is always the bracket to compare.

Min Stack

Purpose

Build a stack that can return the minimum value instantly.

Complexity

Each operation: $O(1)$, Space: $O(n)$

Clean code

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.minstack = []

    def push(self, val: int) -> None:
        self.stack.append(val)

        if not self.minstack:
            self.minstack.append(val)
        else:
            self.minstack.append(min(val, self.minstack[-1]))

    def pop(self) -> None:
        if self.stack:
            self.stack.pop()
            self.minstack.pop()
        else:
            raise IndexError('Stack is Empty!')

    def top(self) -> int:
        if self.stack:
            return self.stack[-1]
        raise IndexError('Stack is Empty!')

    def getMin(self) -> int:
        if self.minstack:
            return self.minstack[-1]
        raise IndexError('Stack is Empty!')
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Constructor	<code>def __init__(self):</code>	Runs when a MinStack object is created.
Main stack	<code>self.stack = []</code>	Stores the actual pushed values.
Minimum stack	<code>self.minstack = []</code>	Stores the minimum value at every stack level.
Push value	<code>self.stack.append(val)</code>	Adds the new value to the normal stack.
First minimum	<code>if not self.minstack: self.minstack.append(val)</code>	If there is no minimum yet, the current value is minimum.
Track minimum	<code>self.minstack.append(min(val, self.minstack[-1]))</code>	Stores the smaller value between current value and previous minimum.
Pop both stacks	<code>self.stack.pop(); self.minstack.pop()</code>	Removes from both stacks so they stay aligned.

Syntax / concept	Actual code line	Plain English meaning
Top value	<code>return self.stack[-1]</code>	Returns the latest pushed value.
Current min	<code>return self.minstack[-1]</code>	Returns the minimum value for the current stack state.
Error handling	<code>raise IndexError(...)</code>	Prevents invalid operations on an empty stack.

Final summary

The normal stack stores values. The minstack stores the minimum at each level. Since the current minimum is always at minstack[-1], getMin is O(1).

Daily Temperatures

Purpose
For each day, find how many days until a warmer temperature appears.

Complexity
Brute: $O(n^2)$. Stack: $O(n)$

Clean code

```
from typing import List

class Solution:
    def dailyTemperaturesBrute(self, temperatures: List[int]) -> List[int]:
        answer = []
        left = 0

        while left < len(temperatures):
            current_temp = temperatures[left]
            count = 0
            found = False
            i = left

            while i < len(temperatures) - 1:
                count += 1
                if current_temp < temperatures[i + 1]:
                    answer.append(count)
                    found = True
                    break
                else:
                    i += 1

            if not found:
                answer.append(0)
                left += 1

        return answer

    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        answer = [0] * len(temperatures)
        stack = []

        for i, temp in enumerate(temperatures):
            while stack and temp > stack[-1][0]:
                old_temp, old_index = stack.pop()
                answer[old_index] = i - old_index

            stack.append((temp, i))

        return answer
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Answer initialized	answer = [0] * len(temperatures)	Starts with 0 for every day because some days may never get warmer.
Stack setup	stack = []	Stores unresolved temperatures and their indexes.
Enumerate loop	for i, temp in enumerate(temperatures):	Gets both index and temperature for each day.

Syntax / concept	Actual code line	Plain English meaning
Warmer check	<code>while stack and temp > stack[-1][0]:</code>	If today is warmer than the unresolved day on top, resolve that old day.
Pop old day	<code>old_temp, old_index = stack.pop()</code>	Removes the old unresolved day because its answer is found.
Days waited	<code>answer[old_index] = i - old_index</code>	Difference between current index and old index is the waiting time.
Store current day	<code>stack.append((temp, i))</code>	Saves today as unresolved until a warmer future day appears.
Return answer	<code>return answer</code>	Returns waiting days for every temperature.
Brute force idea	<code>dailyTemperaturesBrute(...)</code>	Checks each future day one by one, so it works but becomes slow.

Final summary

The monotonic stack keeps days that are still waiting for a warmer future day. When a warmer temperature appears, it pops and resolves all colder previous days.

Generate Parentheses

Purpose

Generate all valid parentheses combinations for n pairs.

Complexity

Catalan growth; recursive generation

Clean code

```
from typing import List

class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        answer = []

        def backtrack(current: str, open_count: int, close_count: int) -> None:
            if len(current) == 2 * n:
                answer.append(current)
                return

            if open_count < n:
                backtrack(current + '(', open_count + 1, close_count)

            if close_count < open_count:
                backtrack(current + ')', open_count, close_count + 1)

        backtrack('', 0, 0)
        return answer
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Answer list	answer = []	Stores every valid completed parentheses string.
Nested helper	def backtrack(current, open_count, close_count):	Recursive function builds strings step by step.
Complete length	if len(current) == 2 * n:	A valid final string must have 2n characters.
Save answer	answer.append(current)	Adds the completed valid string.
Stop branch	return	Stops this recursive path.
Can add opening	if open_count < n:	Opening brackets can be added until n are used.
Add opening	backtrack(current + '(', open_count + 1, close_count)	Try a new path with one more opening bracket.
Can add closing	if close_count < open_count:	Closing bracket is allowed only when there is an unmatched opening bracket.
Add closing	backtrack(current + ')', open_count, close_count + 1)	Try a new path with one more closing bracket.
Start recursion	backtrack('', 0, 0)	Starts with an empty string and zero brackets used.
Return result	return answer	Returns all generated valid combinations.

Final summary

Backtracking tries adding '(' and ')' while obeying rules. It never creates invalid prefixes because it only adds ')' when open_count is greater than close_count.

Largest Rectangle in Histogram

Purpose

Find the largest rectangle area that can be formed in a histogram.

Complexity

Time: O(n), Space: O(n)

Clean code

```
from typing import List

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        stack = []
        max_area = 0

        for index, height in enumerate(heights):
            start_index = index

            while stack and stack[-1][1] > height:
                old_index, old_height = stack.pop()
                width = index - old_index
                area = old_height * width
                max_area = max(max_area, area)
                start_index = old_index

            stack.append((start_index, height))

        for start_index, height in stack:
            width = len(heights) - start_index
            area = height * width
            max_area = max(max_area, area)

        return max_area
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Stack setup	stack = []	Stores pairs of (start_index, height) in increasing height order.
Best area	max_area = 0	Keeps the largest rectangle found so far.
Loop bars	for index, height in enumerate(heights):	Reads each bar with its index and height.
Start position	start_index = index	Assumes current bar starts at its own index first.
Shorter bar found	while stack and stack[-1][1] > height:	A lower bar means taller previous bars must end here.
Pop taller bar	old_index, old_height = stack.pop()	Removes a taller bar and calculates its max rectangle.
Width	width = index - old_index	The popped height extends from old_index up to before current index.
Area	area = old_height * width	Rectangle area using popped height.
Update max	max_area = max(max_area, area)	Keeps the largest area.

Syntax / concept	Actual code line	Plain English meaning
Carry start	<code>start_index = old_index</code>	Current smaller bar can start where the popped taller bar started.
Push current	<code>stack.append((start_index, height))</code>	Stores the current bar with its earliest valid start.
Finalize stack	<code>for start_index, height in stack:</code>	Bars left in stack extend until the end.
Final width	<code>width = len(heights) - start_index</code>	Calculates width to the end of the histogram.
Return max	<code>return max_area</code>	Returns the largest rectangle area.

Final summary

The stack keeps increasing heights. When a smaller height arrives, it proves taller bars cannot extend further, so their rectangle areas are calculated immediately.

TOPIC

Binary Search

Problems 16 to 20

Binary Search

Purpose
Find the target index in a sorted array.

Complexity
Time: $O(\log n)$, Space: $O(1)$

Clean code

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left = 0
        right = len(nums) - 1

        while left <= right:
            middle = (left + right) // 2

            if target < nums[middle]:
                right = middle - 1
            elif target > nums[middle]:
                left = middle + 1
            else:
                return middle

        return -1
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Left boundary	left = 0	Start of search area.
Right boundary	right = len(nums) - 1	End of search area.
Valid search range	while left <= right:	Keep searching while the range still exists.
Middle index	middle = (left + right) // 2	Checks the center of the current range.
Target smaller	if target < nums[middle]: right = middle - 1	Search only the left half.
Target bigger	elif target > nums[middle]: left = middle + 1	Search only the right half.
Target found	return middle	Return the index when the middle value is the target.
Not found	return -1	Return -1 after the search range becomes empty.

Final summary
Binary search repeatedly cuts the sorted search space in half. That is why it is much faster than checking every element.

Search in Rotated Sorted Array

Purpose

Find target in a sorted array that was rotated.

Complexity

Time: $O(\log n)$, Space: $O(1)$

Clean code

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left = 0
        right = len(nums) - 1

        while left <= right:
            middle = (left + right) // 2

            if nums[middle] == target:
                return middle

            if nums[left] <= nums[middle]:
                if nums[left] <= target < nums[middle]:
                    right = middle - 1
                else:
                    left = middle + 1
            else:
                if nums[middle] < target <= nums[right]:
                    left = middle + 1
                else:
                    right = middle - 1

        return -1
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Middle check	<code>if nums[middle] == target:</code>	Return immediately if middle is the target.
Left half sorted	<code>if nums[left] <= nums[middle]:</code>	Detects whether the left side is normally sorted.
Target in left half	<code>if nums[left] <= target < nums[middle]:</code>	If target lies inside the sorted left half, search there.
Move right boundary	<code>right = middle - 1</code>	Cuts off the right half.
Otherwise search right	<code>left = middle + 1</code>	Target must be in the other side.
Right half sorted	<code>else:</code>	If left half is not sorted, the right half must be sorted.
Target in right half	<code>if nums[middle] < target <= nums[right]:</code>	Checks if target lies in sorted right half.
Move left boundary	<code>left = middle + 1</code>	Search the right half.
Move right boundary	<code>right = middle - 1</code>	Search the left half.

Syntax / concept	Actual code line	Plain English meaning
Not found	return -1	Target does not exist in the array.

Final summary

Even though the array is rotated, at least one half around the middle is always sorted. Find the sorted half, check if target is inside it, then discard the other half.

Find Minimum in Rotated Sorted Array

Purpose

Find the smallest value in a rotated sorted array.

Complexity

Time: $O(\log n)$, Space: $O(1)$

Clean code

```
from typing import List

class Solution:
    def findMin(self, nums: List[int]) -> int:
        left = 0
        right = len(nums) - 1

        while left < right:
            middle = (left + right) // 2

            if nums[middle] > nums[right]:
                left = middle + 1
            else:
                right = middle

        return nums[left]
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Search boundaries	left = 0; right = len(nums) - 1	The minimum is somewhere inside this range.
Loop until meet	while left < right:	Stop when both pointers point to the minimum.
Middle index	middle = (left + right) // 2	Checks the middle of the current range.
Middle greater than right	if nums[middle] > nums[right]:	Minimum must be to the right of middle.
Move left	left = middle + 1	Discard middle and everything left of it.
Middle not greater	else: right = middle	Minimum could be middle or to the left, so keep middle.
Return min	return nums[left]	When pointers meet, that index holds the minimum.

Final summary

Compare middle with the right value. If middle is greater, the rotation drop is on the right. Otherwise, the minimum is at middle or on the left.

Koko Eating Bananas

Purpose
Find the minimum eating speed that lets Koko finish within h hours.

Complexity
Time: $O(n \log \max(\text{piles}))$, Space: $O(1)$

Clean code

```
from typing import List

class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        left = 1
        right = max(piles)

        if len(piles) == h:
            return right

        while left < right:
            middle = (left + right) // 2
            total_hours = 0

            for pile in piles:
                hours_per_pile = (pile + middle - 1) // middle
                total_hours += hours_per_pile

            if total_hours > h:
                left = middle + 1
            else:
                right = middle

        return left
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Minimum speed	left = 1	Slowest possible speed is 1 banana per hour.
Maximum speed	right = max(piles)	Fastest useful speed is the largest pile.
Shortcut	if len(piles) == h: return right	If there is one hour per pile, speed must handle the largest pile.
Binary search	while left < right:	Searches for the smallest valid speed.
Try speed	middle = (left + right) // 2	Middle is the current eating speed guess.
Reset hours	total_hours = 0	Counts hours needed at this speed.
Loop piles	for pile in piles:	Checks every pile.
Ceiling division	hours_per_pile = (pile + middle - 1) // middle	Rounds up pile / speed without using math.ceil.
Add hours	total_hours += hours_per_pile	Adds this pile time to total time.

Syntax / concept	Actual code line	Plain English meaning
Too slow	<code>if total_hours > h: left = middle + 1</code>	Need a faster speed.
Valid speed	<code>else: right = middle</code>	This speed works, but maybe a smaller speed also works.
Return speed	<code>return left</code>	Left is the minimum valid speed.

Final summary

The answer is not an index; it is a speed. Binary search tests a speed, calculates hours needed, and narrows toward the smallest speed that works.

Median of Two Sorted Arrays

Purpose

Combine two sorted arrays and return the middle value.

Complexity

Time: $O((m+n) \log(m+n))$, Space: $O(m+n)$

Clean code

```
from typing import List

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        nums1 += nums2
        nums1 = sorted(nums1)

        if len(nums1) % 2 == 0:
            left = 0
            right = len(nums1) - 1
            middle = (left + right) // 2
            median = (nums1[middle] + nums1[middle + 1]) / 2
            return median
        else:
            left = 0
            right = len(nums1) - 1
            middle = (left + right) // 2
            median = nums1[middle]
            return median
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Combine arrays	<code>nums1 += nums2</code>	Adds all values from nums2 into nums1.
Sort merged array	<code>nums1 = sorted(nums1)</code>	Ensures the combined array is in sorted order.
Even length check	<code>if len(nums1) % 2 == 0:</code>	Even length means there are two middle values.
Middle index	<code>middle = (left + right) // 2</code>	Finds the left middle index.
Even median	<code>(nums1[middle] + nums1[middle + 1]) / 2</code>	Average of the two middle values.
Odd median	<code>median = nums1[middle]</code>	For odd length, the single middle value is the median.
Return median	<code>return median</code>	Returns the final median as a number.

Final summary

This is the easy working approach: merge, sort, and take the middle. LeetCode has a harder $O(\log(m+n))$ expected version, but this version clearly explains the median concept.

TOPIC

Sliding Window

Problems 21 to 25

Best Time to Buy and Sell Stock

Purpose

Find the maximum profit from one buy and one sell.

Complexity

Time: O(n), Space: O(1)

Clean code

```
from typing import List

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        left = 0
        right = 1
        max_profit = 0

        while right < len(prices):
            profit = prices[right] - prices[left]

            if profit > 0:
                max_profit = max(max_profit, profit)
            else:
                left = right

            right += 1

        return max_profit
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Buy pointer	left = 0	Left represents the current buying day.
Sell pointer	right = 1	Right represents the current selling day.
Best profit	max_profit = 0	Stores the best profit found so far.
Loop sell days	while right < len(prices):	Try every possible selling day after the buy day.
Profit calculation	profit = prices[right] - prices[left]	Selling price minus buying price.
Profitable trade	if profit > 0:	Only positive profit can improve the answer.
Update best	max_profit = max(max_profit, profit)	Keeps the maximum profit.
Found cheaper buy	else: left = right	If profit is negative or zero, current day is a better buy day.
Move sell pointer	right += 1	Checks the next selling day.
Return result	return max_profit	Returns the best one-transaction profit.

Final summary

The left pointer keeps the cheapest buying day seen so far. The right pointer scans selling days. If a cheaper price appears, left moves there.

Longest Substring Without Repeating Characters

Purpose

Find the longest substring where no character repeats.

Complexity

Time: $O(n)$, Space: $O(n)$

Clean code

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        left = 0
        seen = set()
        max_length = 0

        for right in range(len(s)):
            while s[right] in seen:
                seen.remove(s[left])
                left += 1

            seen.add(s[right])
            max_length = max(max_length, right - left + 1)

        return max_length
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Left boundary	left = 0	Start of the current window.
Seen set	seen = set()	Stores characters currently inside the window.
Best length	max_length = 0	Stores the longest valid window length.
Right loop	for right in range(len(s)):	Expands the window one character at a time.
Duplicate check	while s[right] in seen:	If the new character is already inside, shrink from the left.
Remove left char	seen.remove(s[left])	Removes characters until duplicate is gone.
Move left	left += 1	Shrinks the window.
Add current char	seen.add(s[right])	Adds the new non-duplicate character.
Update length	max_length = max(max_length, right - left + 1)	Calculates current valid window length and updates best.
Return best	return max_length	Returns the longest length found.

Final summary

The window always contains unique characters. When a duplicate enters, the left pointer moves until the duplicate is removed.

Minimum Window Substring

Purpose

Find the smallest substring in *s* that contains all characters of *t*.

Complexity

Time: $O(n)$, Space: $O(k)$

Clean code

```
from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if len(t) > len(s):
            return ''

        need = Counter(t)
        window = {}
        have = 0
        need_count = len(need)
        best = ''
        best_len = float('inf')
        left = 0

        for right in range(len(s)):
            char = s[right]
            window[char] = window.get(char, 0) + 1

            if char in need and window[char] == need[char]:
                have += 1

            while have == need_count:
                current_len = right - left + 1

                if current_len < best_len:
                    best_len = current_len
                    best = s[left : right + 1]

                left_char = s[left]
                window[left_char] -= 1

                if left_char in need and window[left_char] < need[left_char]:
                    have -= 1

                left += 1

        return best
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Impossible case	<code>if len(t) > len(s): return ''</code>	If <i>t</i> is longer, <i>s</i> cannot contain it.
Need counts	<code>need = Counter(t)</code>	Stores required character counts from <i>t</i> .
Window counts	<code>window = {}</code>	Stores character counts in current window.
Have count	<code>have = 0</code>	Number of required characters whose counts are satisfied.

Syntax / concept	Actual code line	Plain English meaning
Need count	<code>need_count = len(need)</code>	Number of unique required characters.
Best answer	<code>best = ""; best_len = float("inf")</code>	Stores the smallest valid window found.
Expand right	<code>for right in range(len(s)):</code>	Adds one character to the window at a time.
Add char count	<code>window[char] = window.get(char, 0) + 1</code>	Updates current window frequency.
Requirement satisfied	<code>if char in need and window[char] == need[char]: have += 1</code>	Marks one required character as satisfied.
Valid window loop	<code>while have == need_count:</code>	When all requirements are met, try shrinking.
Current length	<code>current_len = right - left + 1</code>	Measures the valid window.
Update best	<code>best = s[left : right + 1]</code>	Saves the smaller valid substring.
Remove left	<code>window[left_char] -= 1</code>	Shrinks the window from the left.
Requirement lost	<code>if left_char in need and window[left_char] < need[left_char]: have -= 1</code>	If removing left breaks validity, stop shrinking.
Move left	<code>left += 1</code>	Moves the left edge forward.
Return best	<code>return best</code>	Returns smallest valid substring.

Final summary

Expand right until the window contains all required characters. Then shrink left as much as possible while still valid. Save the smallest valid window seen.

Longest Repeating Character Replacement

Purpose

Find the longest substring that can become all one character after at most k replacements.

Complexity

Time: O(n), Space: O(1) for uppercase alphabet

Clean code

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        key_value = {}
        left = 0
        max_freq = 0
        max_length = 0

        for right in range(len(s)):
            char = s[right]
            key_value[char] = key_value.get(char, 0) + 1
            max_freq = max(max_freq, key_value[char])

            window_length = right - left + 1
            changes_needed = window_length - max_freq

            if changes_needed > k:
                key_value[s[left]] -= 1
                left += 1

            max_length = max(max_length, right - left + 1)

        return max_length
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Frequency map	key_value = {}	Counts characters inside the current window.
Left pointer	left = 0	Start of the current window.
Max frequency	max_freq = 0	Highest count of one character seen in current/previous window.
Best length	max_length = 0	Longest valid window found.
Expand right	for right in range(len(s)):	Adds one character at a time.
Current char	char = s[right]	Gets the entering character.
Update count	key_value[char] = key_value.get(char, 0) + 1	Increases frequency of current character.
Update max freq	max_freq = max(max_freq, key_value[char])	Tracks the most repeated character count.
Window length	window_length = right - left + 1	Current window size.
Changes needed	changes_needed = window_length - max_freq	Characters not equal to majority char must be replaced.

Syntax / concept	Actual code line	Plain English meaning
Too many changes	<code>if changes_needed > k:</code>	Window is invalid if replacements needed exceed k.
Remove left count	<code>key_value[s[left]] -= 1</code>	Shrinks by removing the left character from counts.
Move left	<code>left += 1</code>	Shrinks the window.
Update best	<code>max_length = max(max_length, right - left + 1)</code>	Stores the longest valid window.
Return answer	<code>return max_length</code>	Returns maximum possible length.

Final summary

The trick is `window_length - max_freq`. That tells how many characters must be replaced to make the whole window one repeated character.

Sliding Window Maximum

Purpose

Return the maximum value from every window of size k.

Complexity

Brute: $O(n*k)$. Deque: $O(n)$

Clean code

```
from typing import List
from collections import deque

class Solution:
    def maxSlidingWindowBrute(self, nums: List[int], k: int) -> List[int]:
        list_max = []
        current_window = []

        for right in range(len(nums)):
            if len(current_window) < k:
                current_window.append(nums[right])

            if len(current_window) == k:
                list_max.append(max(current_window))
                current_window.pop(0)

        return list_max

    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        result = []

        for right in range(len(nums)):
            while q and nums[q[-1]] < nums[right]:
                q.pop()

            q.append(right)

            if q[0] <= right - k:
                q.popleft()

            if right >= k - 1:
                result.append(nums[q[0]])

        return result
```

Line-by-line explanation

Syntax / concept	Actual code line	Plain English meaning
Brute window list	<code>current_window = []</code>	Stores the actual current k values for the brute force attempt.
Fill window	<code>current_window.append(nums[right])</code>	Adds the new value into the brute force window.
Window ready	<code>if len(current_window) == k:</code>	When the list has k elements, take its max.
Brute max	<code>list_max.append(max(current_window))</code>	Works logically but max scans k values every time, causing TLE for large k.
Remove old value	<code>current_window.pop(0)</code>	Moves the brute force window, but list pop(0) is also slow.

Syntax / concept	Actual code line	Plain English meaning
Deque setup	<code>q = deque()</code>	Stores indexes of useful max candidates, not the full window.
Result list	<code>result = []</code>	Stores each window maximum.
Right loop	<code>for right in range(len(nums)):</code>	Moves right pointer across the array.
Remove smaller from back	<code>while q and nums[q[-1]] < nums[right]:</code>	If the new value is bigger than the back candidate, the old smaller candidate is useless.
Pop back	<code>q.pop()</code>	Removes from the right/back of the deque.
Append index	<code>q.append(right)</code>	Adds the current index as a possible max candidate.
Expired index check	<code>if q[0] <= right - k:</code>	If the front index is outside the current window, remove it.
Pop front	<code>q.popleft()</code>	Removes the oldest expired index from the left/front.
Window formed	<code>if right >= k - 1:</code>	The first full window is ready when right reaches k - 1.
Append max	<code>result.append(nums[q[0]])</code>	The front of q is always the index of the current max value.
Return result	<code>return result</code>	Returns all sliding window maximums.

Final summary

The brute force version can pass small cases but fails large cases because it recalculates max for every window. The deque version does not track window size; right and k track the real window. The deque only keeps useful maximum candidates, with the biggest value at q[0].